GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura i Esport

IES JUAN DE GARAY
C/ Juan de Garay, 25 46017 Valencia
Tel: 961 20 60 45 Fax: 961 20 60 46
46012963@gva.es www.juandegaray.es

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

UD 06.1. JAVASCRIPT 1

LM

Ciclos Formativos Grado Superior

Pascual Ligero
pa.ligero2@edu.gva.es

2025/2026

Versión:260206.2012

Licencia

Reconocimiento – NoComercial – CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

⚠ Importante

⚛ Interesante

ÍNDICE DE CONTENIDO

1. Introducción a JavaScript ... 6
1.1 Historia y evolución ... 6
1.2 JavaScript vs ECMAScript ... 6
1.3 Evolución del ecosistema ... 6
1.4 JavaScript en el ecosistema web actual ... 7
1.4.1 La tríada del desarrollo web ... 7
1.4.2 Áreas de aplicación ... 7
1.4.3 Frameworks y bibliotecas populares ... 7
1.4.4 Tendencias actuales ... 8
1.5 Configuración del entorno de desarrollo ... 8
1.5.1 Editores de código y IDEs ... 8
1.5.2 Navegadores y herramientas de desarrollo ... 9
1.5.3 Entorno de desarrollo local ... 9
1.5.4 Entornos online ... 10
1.6 Consola del navegador y depuración básica ... 10
1.6.1 Acceso a la consola ... 10
1.6.2 Comandos básicos de consola ... 10
1.6.3 Depuración básica ... 11
1.6.4 Errores comunes y cómo identificarlos ... 11
1.6.5 Herramientas adicionales ... 11
2. Introducción y Entorno ... 12
2.1 Ejemplos de Código ... 12
2.2 Errores Típicos ... 13
2.3 Mini-Reto (3 min) ... 13
3. Variables y Tipos de Datos ... 13
3.1 Tipos Primitivos (La materia prima) ... 13
3.2 Ejemplos de Código ... 13
3.3 Errores Típicos ... 14
3.4 Mini-Reto (4 min) ... 14
4. Operadores ... 14
4.1 Ejemplos de Código ... 15
4.2 Errores Típicos ... 15
4.3 Mini-Reto (3 min) ... 15
5. Control de Flujo ... 15
5.1 Ejemplos de Código ... 16
5.2 Errores Típicos ... 16
5.3 Mini-Reto (5 min) ... 16
6. Cheat Sheet (Lo imprescindible) ... 16
7. Preguntas Rápidas (V/F) ... 17
8. Checklist de Habilidades ... 17
9. Ejercicios Primera parte ... 17
9.1 Nivel 1: Muy Fáciles (Calentamiento) ... 17
9.2 Nivel 2: Medios (Lógica) ... 18
9.3 Nivel 3: Boss Level (Integración) ... 18
10. Funciones Básicas (Declaración y Expresión) ... 19
10.1 Ejemplos de Código ... 19
10.2 Errores Típicos ... 20
10.3 Mini-Reto (3 min) ... 20
11. Arrow Functions (Flecha) => ... 20
11.1 Ejemplos de Código ... 20
11.2 Errores Típicos ... 21
11.3 Mini-Reto (3 min) ... 21
12. Parámetros (Default y Rest) ... 21
12.1 Ejemplos de Código ... 21
12.2 Errores Típicos ... 22
12.3 Mini-Reto (4 min) ... 22

13. Scope y Hoisting **22**
13.1 Ejemplos de Código 23
13.2 Errores Típicos 23
13.3 Mini-Reto (4 min) 23
14. Callbacks (Funciones como argumentos) **23**
14.1 Ejemplos de Código 24
14.2 Errores Típicos 24
14.3 Mini-Reto (5 min) 24
15. Cheat Sheet Parte 2 ..... **24**
16. Preguntas Rápidas (V/F) **25**
17. Checklist de Habilidades **25**
18. Ejercicios PARTE 2 ..... **25**
18.1 Nivel 1: Muy Fáciles 25
18.2 Nivel 2: Medios (Lógica Modular) 26
18.3 Nivel 3: Boss Level (Integración) 26
19. Bibliografía **27**

UD06.1. JAVASCRIPT 1

1. INTRODUCCIÓN A JAVASCRIPT

1.1 Historia y evolución

JavaScript fue creado en 1995 por Brendan Eich mientras trabajaba en Netscape Communications Corporation. Originalmente llamado Mocha, luego LiveScript, y finalmente JavaScript, fue desarrollado en solo 10 días como un lenguaje de scripting para el navegador Netscape Navigator.

A pesar de su nombre, JavaScript no está relacionado directamente con Java. El nombre fue principalmente una estrategia de marketing, ya que Java era muy popular en ese momento. En 1997, se presentó a ECMA International para su estandarización, dando lugar a ECMAScript.

La evolución de JavaScript ha sido notable: - ECMAScript 1 (1997): Primera edición - ECMAScript 3 (1999): Añadió expresiones regulares, manejo de excepciones - ECMAScript 5 (2009): Añadió strict mode, JSON, métodos de array - ECMAScript 6/ES2015 (2015): Una actualización masiva que introdujo clases, módulos, promesas, arrow functions, let/const - Desde 2015, ECMAScript adopta un ciclo de lanzamiento anual (ES2016, ES2017, etc.)

Hoy, JavaScript es el lenguaje de programación más utilizado en el desarrollo web, ejecutándose no solo en navegadores sino también en servidores (Node.js), aplicaciones móviles, y más.

1.2 JavaScript vs ECMAScript

Es importante entender la diferencia entre JavaScript y ECMAScript:

- **ECMAScript**: Es la especificación estándar del lenguaje. Define la sintaxis, tipos, declaraciones, y otros aspectos fundamentales.
- **JavaScript**: Es la implementación más conocida de ECMAScript. Incluye todo lo definido por ECMAScript más características adicionales específicas para navegadores como el DOM (Document Object Model) y BOM (Browser Object Model).

Cuando hablamos de ES6, ES2015, etc., nos referimos a versiones específicas de la especificación ECMAScript que JavaScript implementa.

```
// Ejemplo de característica ES6: Arrow Functions
// Antes de ES6
var suma = function(a, b) {
  return a + b;
};
```

```
// Con ES6
const suma = (a, b) => a + b;
```

1.3 Evolución del ecosistema

JavaScript ha evolucionado de ser un simple lenguaje para validación de formularios a convertirse en un ecosistema completo:

- **Node.js** (2009): Llevó JavaScript al lado del servidor
- **Frameworks frontend**: Angular, React, Vue.js
- **Herramientas de construcción**: Webpack, Babel, Vite
- **TypeScript** (2012): Superset de JavaScript con tipado estático
- **Deno** (2018): Runtime alternativo a Node.js
- **WebAssembly**: Permite ejecutar código de alto rendimiento en navegadores

1.4 JavaScript en el ecosistema web actual

JavaScript es actualmente uno de los lenguajes más populares y versátiles, formando parte fundamental del desarrollo web moderno.

1.4.1 La tríada del desarrollo web

- **HTML**: Estructura del contenido
- **CSS**: Presentación y estilo
- **JavaScript**: Comportamiento e interactividad

1.4.2 Áreas de aplicación

1. **Desarrollo Frontend**
   - Manipulación del DOM
   - Gestión de eventos de usuario
   - Validación de formularios
   - Animaciones y efectos visuales
   - Comunicación con APIs
   - Aplicaciones de una sola página (SPA)
2. **Desarrollo Backend**
   - Servidores con Node.js
   - APIs RESTful
   - Bases de datos (MongoDB, PostgreSQL)
   - Autenticación y autorización
   - WebSockets para comunicación en tiempo real
3. **Desarrollo Móvil**
   - React Native
   - Ionic
   - NativeScript
   - Capacitor
4. **Desarrollo de Escritorio**
   - Electron
   - NW.js
5. **Internet de las Cosas (IoT)**
   - Johnny-Five

- Node-RED

1.4.3 Frameworks y bibliotecas populares

- **Frontend**:
  - React: Biblioteca para interfaces de usuario
  - Angular: Framework completo desarrollado por Google
  - Vue.js: Framework progresivo
  - Svelte: Compilador en lugar de framework tradicional
- **Backend**:
  - Express.js: Framework minimalista para Node.js
  - Nest.js: Framework inspirado en Angular
  - Fastify: Alternativa de alto rendimiento a Express
- **Fullstack**:
  - Next.js: Framework React con renderizado del lado del servidor
  - Nuxt.js: Equivalente para Vue.js
  - Remix: Framework moderno para aplicaciones web

1.4.4 Tendencias actuales

- **JAMstack**: JavaScript, APIs y Markup
- **Serverless**: Funciones como servicio
- **PWA**: Aplicaciones web progresivas
- **Micro-frontends**: Arquitectura modular para aplicaciones grandes
- **Web Components**: Componentes web reutilizables y encapsulados

1.5 Configuración del entorno de desarrollo

Para comenzar a programar en JavaScript, necesitamos configurar un entorno de desarrollo adecuado.

1.5.1 Editores de código y IDEs

Existen varias opciones excelentes para desarrollar con JavaScript:

1. **Visual Studio Code (VS Code)**
   - Editor gratuito y de código abierto desarrollado por Microsoft
   - Extensiones recomendadas:
     - ESLint: Para análisis de código
     - Prettier: Para formateo de código
     - JavaScript (ES6) code snippets
     - Live Server: Para ejecutar páginas web localmente
   - Instalación: Descargar desde code.visualstudio.com
2. **WebStorm**
   - IDE completo desarrollado por JetBrains
   - Incluye muchas características sin necesidad de extensiones
   - *De pago*, pero con prueba gratuita
3. **Sublime Text**
   - Editor ligero y rápido
   - Altamente personalizable
   - *De pago*, pero con evaluación ilimitada
4. **Atom**
   - Editor gratuito y de código abierto

- Desarrollado por GitHub (ahora parte de Microsoft)

1.5.2 Navegadores y herramientas de desarrollo

Los navegadores modernos incluyen potentes herramientas de desarrollo:

1. **Google Chrome**
   - DevTools: Presiona F12 o Ctrl+Shift+I (Cmd+Option+I en Mac)
   - Secciones principales:
     - Elements: Inspeccionar y modificar HTML/CSS
     - Console: Ejecutar JavaScript y ver mensajes
     - Sources: Depurar código
     - Network: Monitorear solicitudes de red
     - Performance: Analizar rendimiento
2. **Mozilla Firefox**
   - Firefox Developer Edition: Versión específica para desarrolladores
   - Similar a Chrome DevTools con algunas características únicas
3. **Microsoft Edge**
   - Basado en Chromium, herramientas similares a Chrome

1.5.3 Entorno de desarrollo local

Para proyectos simples:

1. **Estructura básica de carpetas**:

```
mi-proyecto/
├── index.html
├── css/
│   └── styles.css
└── js/
    └── script.js
```

2. **Servidor local**:
   - Extensión Live Server en VS Code
   - Servidor Python simple: python -m http.server
   - Node.js con http-server: npx http-server

Para proyectos más complejos:

1. **Node.js y npm (se ve en segundo)**:
   - Instalación: Descargar desde nodejs.org
   - Verificar instalación:

     ```
     node -v
     npm -v
     ```
2. **Gestión de dependencias**:
   - Inicializar proyecto:

     ```
     npm init -y
     ```
   - Instalar paquetes:

     ```
     npm install nombre-paquete
     ```
3. **Herramientas de construcción**:
   - **Vite**: Configuración rápida para proyectos modernos

     ```
     npm create vite@latest mi-app
     ```

- **Webpack**: Empaquetador más configurable
- **Parcel**: Empaquetador de configuración cero

1.5.4 Entornos online

Para experimentación rápida sin configuración local:

1. **CodePen**: codepen.io
2. **JSFiddle**: jsfiddle.net
3. **CodeSandbox**: codesandbox.io
4. **StackBlitz**: stackblitz.com
5. **Replit**: replit.com

1.6 Consola del navegador y depuración básica

La consola del navegador es una herramienta esencial para el desarrollo y depuración de JavaScript.

1.6.1 Acceso a la consola

- **Chrome/Edge**: F12 o Ctrl+Shift+J (Cmd+Option+J en Mac)
- **Firefox**: F12 o Ctrl+Shift+K (Cmd+Option+K en Mac)
- **Safari**: Cmd+Option+C (requiere habilitar menú Desarrollador)

1.6.2 Comandos básicos de consola

1. **Mostrar mensajes**:
```
console.log("Hola mundo"); // Mensaje normal
console.info("Información"); // Mensaje informativo
console.warn("Advertencia"); // Advertencia (amarillo)
console.error("Error"); // Error (rojo)
```
2. **Agrupar mensajes**:
```
console.group("Grupo de mensajes");
console.log("Mensaje 1");
console.log("Mensaje 2");
console.groupEnd();
```
3. **Tablas**:
```
const personas = [
  { nombre: "Ana", edad: 28 },
  { nombre: "Juan", edad: 32 }
];
console.table(personas);
```
4. **Tiempo de ejecución**:
```
console.time("Operación");
// Código a medir
console.timeEnd("Operación");
```
5. **Contadores**:
```
console.count("Contador");
console.count("Contador");
console.countReset("Contador");
```

1.6.3 Depuración básica

1. **Puntos de interrupción (breakpoints)**:
   - En el código: debugger;
   - En DevTools: Clic en el número de línea en la pestaña Sources/Debugger
2. **Inspección de variables**:
   - Hover sobre variables durante la depuración
   - Panel de ámbito (Scope) en DevTools
   - Consola durante la pausa
3. **Control de ejecución**:
   - Continuar (F8): Continúa la ejecución hasta el siguiente breakpoint
   - Step over (F10): Ejecuta la siguiente línea sin entrar en funciones
   - Step into (F11): Entra en la función que se está llamando
   - Step out (Shift+F11): Sale de la función actual
4. **Puntos de observación (watch)**:
   - Añadir expresiones para monitorear en el panel Watch

1.6.4 Errores comunes y cómo identificarlos

1. **Tipos de errores**:
   - **SyntaxError**: Error en la escritura del código
   - **ReferenceError**: Referencia a variable no definida
   - **TypeError**: Operación en un tipo incorrecto
   - **RangeError**: Valor fuera de rango permitido
2. **Lectura de errores**:
   - Mensaje de error: Descripción del problema
   - Nombre del archivo: Dónde ocurrió
   - Número de línea y columna: Ubicación exacta
   - Stack trace: Secuencia de llamadas hasta el error
3. **Consejos para depuración**:
   - Leer el mensaje de error cuidadosamente
   - Verificar la consola frecuentemente durante el desarrollo
   - Usar console.log() estratégicamente
   - Aislar el problema dividiendo el código en partes más pequeñas

1.6.5 Herramientas adicionales

1. **Pestaña Network**:
   - Monitorear solicitudes HTTP
   - Verificar respuestas de API
   - Analizar tiempos de carga
2. **Pestaña Elements**:
   - Inspeccionar el DOM
   - Modificar HTML/CSS en tiempo real
   - Ver eventos asociados a elementos
3. **Pestaña Application**:
   - Gestionar almacenamiento (localStorage, cookies)
   - Verificar service workers
   - Inspeccionar caché

2. INTRODUCCIÓN Y ENTORNO

JavaScript es el cerebro de una página web. Si HTML es el esqueleto y CSS la ropa, JS son los músculos y el sistema nervioso.

- **El Motor:** Tu navegador (Chrome, Edge, Brave) tiene un "motor" dentro (el más famoso es **V8**) que lee tu código y lo ejecuta al instante.
- **La Consola:** Es tu panel de control secreto. Aquí veremos errores y probaremos código. (Clic derecho en la web -> Inspeccionar -> Pestaña "Console").
- **Vincular JS:** Cuando cargas una web, el navegador lee de arriba a abajo.
    - **Sin atributos:** El navegador se detiene, descarga el JS, lo ejecuta y luego sigue pintando la web. (Malo: web lenta).
    - **async:** Descarga el JS mientras pinta la web, pero lo ejecuta en cuanto termina de descargar, pausando el pintado. (Caótico si tienes varios archivos).
    - **defer:** (El ganador). Descarga el JS en paralelo sin molestar y **espera** a que todo el HTML esté listo para ejecutarse.

2.1 Ejemplos de Código

A. HTML Mínimo (Cómo conectar)

HTML

```html
<!DOCTYPE html>
<html>
<head>
    <script src="app.js" defer></script>
</head>
<body>
    <h1>Mi Web App</h1>
</body>
</html>
```

B. Tu primer Debug (app.js)

JavaScript

```javascript
// console.log: Imprime un mensaje informativo (color blanco/negro)
console.log("Iniciando sistema... ");

// console.warn: Imprime una advertencia (color amarillo)
```

```
console.warn("Cuidado: Batería baja");

// console.error: Imprime un error crítico (color rojo)
console.error("Error: No se pudo conectar al servidor");
```

2.2 Errores Típicos

- **Poner el script sin atributos en el <head>:** El navegador se detiene a leer el JS y la pantalla se queda blanca hasta que termine. **Solución:** Usa defer.
- **No encontrar el archivo:** Error "404 Not Found" en consola. **Solución:** Revisa que el nombre app.js coincida exactamente (mayúsculas/minúsculas).

2.3 Mini-Reto (3 min)

Abre la consola de tu navegador ahora mismo. Escribe un console.error("Hackeo fallido") y presiona Enter.

- *Pista: Debe salir en rojo.*

3. VARIABLES Y TIPOS DE DATOS

Una variable es una caja donde guardas información. Antes se usaba var, pero era una caja insegura (se rompía o se cambiaba sola). Hoy usamos const y let.

- **const:** Una caja blindada. Una vez guardas algo, no puedes cambiar el contenido. Úsala el 95% de las veces.
- **let:** Una caja abierta. Puedes cambiar lo que hay dentro. Úsala solo si el valor va a cambiar (puntos, vidas, contador).
- **Scope (Ámbito):** Las variables creadas con let/const dentro de un bloque { ... } (como un if) no existen fuera de él.

3.1 Tipos Primitivos (La materia prima)

1. **String:** Texto ("Hola").
2. **Number:** Números (25, 3.14).
3. **Boolean:** Interruptor (true, false).
4. **Null:** Caja vacía a propósito.
5. **Undefined:** Caja que nadie ha llenado aún (JS lo pone por defecto).

6. **Symbol:** Identificador único (avanzado).
7. **BigInt:** Para números gigantescos (ES2023).

3.2 Ejemplos de Código

JavaScript

```
// Perfil de Usuario
const username = "MidudevFan"; // String
let edad = 25;              // Number
const isPremium = true;       // Boolean
let descuento = null;         // Null (definido explícitamente)

// Cambio de valor
edad = 26; // Correcto, usamos let
username = "OtroUser"; // Error: Assignment to constant variable

// Tipos raros
console.log(typeof username); // "string"
console.log(typeof undefined); // "undefined"
console.log(typeof null); // "object" (Un error histórico de JS, acéptalo y sigue)

// BigInt (Fíjate en la 'n' al final)
const visitasYoutube = 9007199254740991n;
const nuevaVisita = 1n;
console.log(visitasYoutube + nuevaVisita); // 9007199254740992n
```

3.3 Errores Típicos

- **Mezclar Tipos Numéricos:** 10n + 5. **Solución:** JS no mezcla BigInt con Number. Usa 10n + 5n.
- **Usar var: Solución:** Bórralo de tu memoria. var ignora los bloques {} y causa bugs difíciles de rastrear.

3.4 Mini-Reto (4 min)

Crea una variable const llamada dinero con valor 100. Intenta restarle 50 en la línea siguiente (dinero = dinero - 50). Mira el error en consola. Luego arréglalo cambiando la declaración.

4. OPERADORES

Las herramientas para manipular tus variables.

- **Aritméticos:** + - * / % (resto).
- **Asignación:** += (suma y asigna), -= (resta y asigna).
- **Comparación:**
  - == (Comparación débil): "5" es igual a 5. (**Peligroso**, JS convierte tipos a escondidas).
  - === (Comparación estricta): "5" NO es igual a 5. (Seguro, compara valor Y tipo).
- **Lógicos:**
  - && (AND): Todo debe ser verdad (Entrada Y DNI).
  - || (OR): Al menos uno es verdad (Efectivo O Tarjeta).
  - ! (NOT): Invertir (Si NO está lleno).

4.1 Ejemplos de Código

JavaScript

```
// Aritmética gamer
let vida = 100;
vida -= 20; // Recibes daño (vida = 80)
console.log("Vida restante:", vida);

// El peligro de la Coerción
console.log(10 == "10");  // true (JS convierte el texto a número)
console.log(10 === "10"); // false ( Correcto, son tipos distintos)

// Lógica de Discoteca
const edad = 17;
const tienePaseVIP = true;

// Puede pasar si es mayor de 18 O si tiene pase VIP
const puedePasar = (edad >= 18) || tienePaseVIP;

console.log("¿Entra al club?", puedePasar); // true
```

4.2 Errores Típicos

- **Usar = en un if:** if (vida = 0). **Solución:** Eso asigna valor. Para comparar usa ===.
- **Comparar Strings con mayúsculas:** "Hola" === "hola" es false.

4.3 Mini-Reto (3 min)

Tienes const bateria = 15 y const cargando = false. Escribe una condición lógica en un console.log que devuelva true solo si la batería es menor a 20 Y NO está cargando.

- *Pista: Usa < y !.*

5. CONTROL DE FLUJO

Cómo tomar decisiones. "Si pasa esto, haz aquello".

- **if / else:** El semáforo básico.
- **switch:** Útil cuando tienes muchos casos específicos (menú de opciones).
- **Ternario:** Un if/else en una sola línea. condición ? verdadero : falso.
- **Bucles (Repeticiones):**
  - for...of: El mejor para listas (arrays).
  - for...in: Para leer las propiedades de un objeto.
  - while: Repetir mientras algo sea verdad.

5.1 Ejemplos de Código

JavaScript

```
// 1. Ternario (Ideal para asignar valores rápidos)
const saldo = 5;
const mensaje = saldo > 0 ? "Comprar chuches." : "Ahorrar.";
console.log(mensaje);

// 2. Bucle for...of (Recorrer listas)
const playlist = ["Despacito", "Flowers", "Bizarrap"];

console.log("--- Mi Playlist ---");
for (const cancion of playlist) {
    console.log("Reproduciendo: " + cancion);
}
```

```
// 3. Bucle for...in (Recorrer Objetos)
const personaje = { nombre: "Mario", clase: "Fontanero", nivel: 10 };

for (const propiedad in personaje) {
    // Imprime clave -> valor (ej: nombre -> Mario)
    console.log(propiedad + " -> " + personaje[propiedad]);
}
```

5.2 Errores Típicos

- **Bucles Infinitos en while:** Si la condición nunca cambia a false, el navegador se congela.
- **Usar for...in para arrays:** Funciona, pero puede dar problemas de orden o propiedades extra. Usa for...of para listas.

5.3 Mini-Reto (5 min)

Tienes const notas = [10, 5, 8, 4].

Usa un bucle for...of. Dentro, usa un if. Si la nota es mayor o igual a 5, imprime "Aprobado", si no, "Suspenso".

6. CHEAT SHEET (LO IMPRESCINDIBLE)

- **Declarar:** const (siempre), let (si cambia). Nunca var.
- **Comparar:** === (siempre), !== (distinto estricto).
- **Logs:** console.log() para ver qué pasa.
- **Tipos:** typeof variable te dice qué es.
- **Iterar Array:** for (const item of lista) { ... }.
- **Script:** <script src="..." defer></script>.

7. PREGUNTAS RÁPIDAS (V/F)

1. (V/F) defer hace que el script se ejecute antes que el HTML se cargue.
2. (V/F) const permite modificar el valor de la variable más tarde.
3. (V/F) undefined significa que la variable ha sido declarada pero no tiene valor.
4. (V/F) El operador === compara valor y tipo de dato.

5. (V/F) for...in es el bucle recomendado para recorrer arrays (listas).
6. (V/F) 10 == "10" devuelve true.
7. (V/F) BigInt se usa para números decimales pequeños.
8. (V/F) let tiene scope de bloque (solo vive dentro de las llaves {}).
9. (V/F) El motor de JS en Chrome se llama V8.
10. (V/F) El operador ! invierte un valor booleano (!true es false).

8. CHECKLIST DE HABILIDADES

- [ ] Sé vincular un archivo JS al HTML sin bloquear la carga (defer).
- [ ] Sé usar la consola para encontrar errores.
- [ ] Entiendo por qué usar const por defecto.
- [ ] Sé diferenciar null de undefined.
- [ ] Evito usar == y uso siempre ===.
- [ ] Puedo crear un bucle que recorra una lista de canciones o productos.

9. EJERCICIOS PRIMERA PARTE

Restricciones: Solo usa la consola. No uses funciones ni arrays complejos todavía.

9.1 Nivel 1: Muy Fáciles (Calentamiento)

1. **La calculadora rota:** Crea dos variables const con números y una con un String ("10"). Intenta sumarlas y mira el resultado en consola.
2. **El nombre perdido:** Declara una variable let nombre; (sin valor). Imprime su tipo con typeof. Luego asígnale "TuNombre" y vuelve a imprimir el tipo.
3. **Comparador estricto:** Imprime en consola el resultado de comparar 0 === false y 0 == false. Explica la diferencia en un comentario.
4. **Ternario simple:** Crea let edad = 20. Usa un ternario para guardar en let status el texto "Adulto" o "Menor". Imprímelo.
5. **Concatenación:** Crea const saludo = "Hola" y const mundo = "Mundo". Únelos en una variable frase con un espacio en medio.

6. **Resto:** Calcula el resto de dividir 10 entre 3 usando el operador %.

9.2 Nivel 2: Medios (Lógica)

7. **Semáforo Switch:** Crea let color = "rojo". Usa un switch que imprima "Stop" si es rojo, "Precaución" si amarillo, "Go" si verde, y "Error" si es otro.
8. **El Portero de Discoteca:** Tienes edad, tieneEntrada (bool) y esVip (bool). Crea un if que deje pasar si: (Es mayor de edad Y tiene entrada) O (Es Vip).
9. **Contador While:** Usa un bucle while que imprima "Cargando..." 5 veces (usa un contador i que empiece en 0 y suba hasta 5).
10. **Buscador de pares:** Crea un bucle for clásico (o investiga cómo hacerlo) del 1 al 10. Dentro, usa un if y el operador % para imprimir solo los números pares.
11. **Recorrer Objeto:** Crea un objeto const coche = { marca: "Toyota", modelo: "Corolla", año: 2020 }. Usa for...in para imprimir "marca: Toyota", etc.
12. **Login Simulado:** Tienes const userBBDD = "admin" y const passBBDD = "1234". Crea variables inputUser e inputPass. Comprueba con un if si AMBAS coinciden.

9.3 Nivel 3: Boss Level (Integración)

13. La Caja Registradora:

- Tienes const precios = [10, 20, 50].
- Crea una variable let total = 0.
- Usa un bucle for...of para recorrer los precios.
- En cada vuelta, suma el precio al total.
- Al final, imprime "Total a pagar: [total]".
- *Pista: Usa el operador +=.*

14. El Validador de Password:

- Tienes const pass = "secreto123".
- Verifica 3 cosas (inventa las condiciones con if):
    1. Longitud mayor a 5 (usa pass.length).
    2. No es igual a "1234".
    3. Existe (no es null ni undefined).

- Si cumple TODO, imprime "Password segura". Si no, "Password débil".
- *Pista: Usa operadores lógicos &&.*

15.BigInt vs Number (El Crash):

- Crea const maxSafe = Number.MAX_SAFE_INTEGER (el número más grande seguro en JS).
- Imprímelo. Súmale 1 e imprime. Súmale 2 e imprime. (Observa que falla).
- Ahora haz lo mismo pero convirtiéndolo a BigInt (BigInt(maxSafe)).
- Demuestra que con BigInt la suma sí funciona correctamente.

10. FUNCIONES BÁSICAS (DECLARACIÓN Y EXPRESIÓN)

Una función es una **mini-máquina**: le metes datos (inputs/argumentos), hace algo dentro y te devuelve un resultado (output/return). Es como un "hechizo" que guardas y lanzas cuando quieres.

Existen dos formas principales de crearlas:

1. **Declaración de Función (Clásica):** Empieza con function. Se puede usar antes de definirla (gracias al *Hoisting*, que veremos luego).
2. **Expresión de Función:** Guardas una función anónima dentro de una variable const. Es más ordenada porque te obliga a definirla antes de usarla.

10.1 Ejemplos de Código

JavaScript

```
// A. Declaración (Clásica)
function calcularPropina(totalCuenta) {
    return totalCuenta * 0.10; // 10% propina
}

// B. Expresión (Guardada en variable)
const ganarExperiencia = function(puntosActuales, puntosGanados) {
    const total = puntosActuales + puntosGanados;
    console.log("¡Has subido de nivel! XP Total: " + total);
    return total;
};
```

```
// Usarlas (Llamar a la máquina)
const propina = calcularPropina(50);
console.log("Propina a dejar:", propina); // 5

const miXP = ganarExperiencia(100, 50); // Imprime el log y guarda 150 en miXP
```

10.2 Errores Típicos

- **Olvidar el return:** Si no pones return, la función devuelve undefined por defecto. console.log solo muestra, no devuelve valor.
- **Confundir parámetro y argumento:** Parámetro es la variable en la definición (totalCuenta). Argumento es el valor real que envías (50).

10.3 Mini-Reto (3 min)

Crea una función llamada saludarUsuario que reciba un nombre. Debe devolver el string "Bienvenido de nuevo, [nombre]". Pruébala con tu nombre.

11. ARROW FUNCTIONS (FLECHA) =>

La sintaxis moderna (ES6). Es más corta, más limpia y la verás en el 90% del código profesional actual (React, Angular, etc.).

- **Sintaxis:** Quitas la palabra function y usas la flecha =>.
- **Return Implícito:** Si la función hace una sola cosa en una línea, te puedes ahorrar las llaves {} y la palabra return. ¡Magia!
- **El this (Intro):** Las Arrow Functions no crean su propio contexto (this). Toman el de fuera.

11.1 Ejemplos de Código

JavaScript

```
// Función Flecha Clásica
const sumar = (a, b) => {
    return a + b;
};

// Función Flecha PRO (Return implícito)
```

```
// Se lee: "La función restar recibe a y b, y devuelve a - b"
const restar = (a, b) => a - b;

console.log(restar(10, 5)); // 5

// Caso donde NO usar Arrow (Métodos de objeto con 'this')
const usuario = {
	nombre: "Ana",
	saludarMal: () => console.log("Hola " + this.nombre), // Error: this no es el usuario
	saludarBien: function() { console.log("Hola " + this.nombre) } // Correcto
};

usuario.saludarBien(); // "Hola Ana"
```

11.2 Errores Típicos

- **Poner {} y olvidar return en one-liners:** const d = (n) => { n * 2 } devuelve undefined. O quitas las llaves o pones return.
- **Usar Arrow methods para todo:** Si necesitas acceder al propio objeto con this, usa la función normal.

11.3 Mini-Reto (3 min)

Convierte estas 3 funciones a Arrow Functions (intenta usar return implícito donde puedas):

1. function doble(n) { return n * 2; }
2. function saludo() { return "Hola"; }
3. function sumar(a, b) { let total = a + b; return total; } (Ojo, esta tiene varias líneas).

12. PARÁMETROS (DEFAULT Y REST)

Haciendo tus funciones más inteligentes y flexibles.

- **Parámetros por defecto (param = valor):** Si el usuario no envía el dato, usa uno de reserva. Evita errores undefined y miles de if.
- **Rest Operator (...args):** "El resto de los argumentos". Permite recibir una cantidad infinita de datos y los mete en un array (lista).

12.1 Ejemplos de Código

JavaScript

```
// 1. Parámetros por defecto (Default)
// Si no me das usuario, serás "Invitado". Si no hay tag, será "General".
const crearMensaje = (usuario = "Invitado", etiqueta = "General") => {
    console.log(`[${etiqueta}] Usuario: ${usuario}`);
};

crearMensaje("Carlos", "Admin"); // "[Admin] Usuario: Carlos"
crearMensaje();                  // "[General] Usuario: Invitado"

// 2. Rest Operator (...numeros)
// No sé cuántos números vas a enviar, así que los atrapo todos en un array
const sumarTodo = (...numeros) => {
    console.log(numeros); // Muestra [10, 20, 30, 50]

    let total = 0;
    for (const num of numeros) { // Usamos for...of del Módulo 1
        total += num;
    }
    return total;
};

console.log(sumarTodo(10, 20, 30, 50)); // 110
```

12.2 Errores Típicos

- **Poner el Rest al principio:** function mal(...resto, a). **Corrección:** El ...resto siempre debe ser el **último** parámetro.
- **Confundir con Spread:** El operador es el mismo (...), pero aquí *agrupa* argumentos en una lista. Spread *esparce* una lista en argumentos (lo veremos luego).

12.3 Mini-Reto (4 min)

Crea una función calcularPromedio(...notas) que reciba cualquier cantidad de notas, las sume y devuelva la media.

- *Pista: Necesitas saber cuántas notas hay. Usa notas.length.*

13. SCOPE Y HOISTING

¿Dónde viven tus variables? Entender esto evita bugs invisibles.

- **Scope (Alcance):** Es la zona donde una variable "existe".
  - **Global:** Accesible desde cualquier sitio. (Peligroso, cualquiera la toca).
  - **Local/Bloque:** Variables creadas dentro de { ... } (if, for, function). Solo existen ahí dentro. Es como el área VIP: lo que pasa dentro, se queda dentro.
- **Hoisting (Elevación):** JS mueve las declaraciones al inicio antes de ejecutar.
  - function se eleva completa (puedes usarla antes de escribirla).
  - let/const se "elevan" pero a una "Zona Muerta" (TDZ). Si las usas antes, explota con error.

13.1 Ejemplos de Código

JavaScript

```
// SCOPE
const secreto = "12345"; // Global

if (true) {
    const claveLocal = "Abcde"; // Local (Bloque)
    console.log(secreto); // Funciona (lee de fuera)
}

// console.log(claveLocal); // ¡ERROR! ReferenceError: claveLocal is not defined

// HOISTING
console.log(saludarClasico()); // Funciona (se eleva)
function saludarClasico() { return "Hola"; }

// console.log(saludarModerno()); // ¡ERROR! No puedes acceder antes de inicializar
const saludarModerno = () => "Hola";
```

13.2 Errores Típicos

- **Declarar todo global:** Llenar el archivo de variables sueltas. **Corrección:** Mete la lógica dentro de funciones.
- **Shadowing confuso:** Crear una variable local con el mismo nombre que una global. JS usará la local, pero te confundirá a ti.

13.3 Mini-Reto (4 min)

Mira este código. ¿Por qué falla? Arréglalo moviendo o cambiando lo mínimo posible.

JavaScript

```
console.log(mensaje);
let mensaje = "Hola mundo";
```

14. CALLBACKS (FUNCIONES COMO ARGUMENTOS)

En JS, las funciones son "ciudadanos de primera clase". Significa que puedes pasar una función como si fuera un número o un texto a otra función.

- **Callback:** Una función que pasas como argumento a otra para que la ejecute después.
- **Uso:** "Cuando termines de procesar estos datos, llama a esta función". Es la base de la interactividad (clicks, eventos) y la asincronía.

14.1 Ejemplos de Código

JavaScript

```
// Función Maestra (procesador)
const procesarUsuarios = (listaUsuarios, accionCallback) => {
    for (const usuario of listaUsuarios) {
        // Ejecutamos la función que nos pasaron para cada usuario
        accionCallback(usuario);
    }
};

const misAmigos = ["Ana", "Beto", "Carla"];

// 1. Callback definido fuera
const gritarNombre = (nombre) => console.log("¡" + nombre.toUpperCase() + "!");

procesarUsuarios(misAmigos, gritarNombre);

// 2. Callback anónimo (Arrow Function directa)
// Muy común: definir la función ahí mismo
procesarUsuarios(misAmigos, (nombre) => {
    console.log("Usuario procesado: " + nombre);
});
```

14.2 Errores Típicos

- **Ejecutar el callback al pasarlo:** procesar(lista, gritar()). **Corrección:** Pasa la referencia sin paréntesis: procesar(lista, gritar). Si pones (), se ejecuta al instante y pasas el resultado (undefined), no la función.

14.3 Mini-Reto (5 min)

Crea una función repetir(n, accion) que reciba un número y un callback. Debe ejecutar el callback n veces.

Llama a repetir(3, () => console.log("Hola")).

15. CHEAT SHEET PARTE 2

- **Función:** const nombre = (a, b) => { ... }.
- **Return Implícito:** const suma = (a, b) => a + b.
- **Parámetros:** (usuario = "Invitado") (Default) y (...numeros) (Rest).
- **Scope:** Variables en {} mueren al salir. let/const no se pueden usar antes de declarar.
- **Callback:** Pasar una función fn a otra: hacerAlgo(datos, fn).

16. PREGUNTAS RÁPIDAS (V/F)

1. (V/F) Una Arrow Function siempre necesita la palabra return explícita.
2. (V/F) Las funciones declaradas con function se pueden usar antes de definirlas (Hoisting).
3. (V/F) El parámetro Rest ...args debe ir siempre al final.
4. (V/F) Las variables const dentro de un if son accesibles desde fuera.
5. (V/F) Un callback es una función que se pasa como argumento.
6. (V/F) Si no paso valor a un parámetro con default, vale undefined.
7. (V/F) Arrow functions tienen su propio this.
8. (V/F) const sumar = (a, b) => a + b devuelve la suma automáticamente.
9. (V/F) Es buena práctica usar variables globales para todo.

10.(V/F) Puedo pasar una función anónima directamente como argumento.

17. CHECKLIST DE HABILIDADES

- [ ] Sé convertir una función clásica en Arrow Function.
- [ ] Entiendo cuándo puedo omitir las llaves {} y el return.
- [ ] Sé usar parámetros por defecto para evitar errores.
- [ ] Entiendo por qué una variable declarada dentro de una función no existe fuera.
- [ ] Puedo crear una función que acepte otra función (callback) y la ejecute.

18. EJERCICIOS PARTE 2

Restricciones: Usa consola. Intenta usar Arrow Functions siempre que sea posible.

18.1 Nivel 1: Muy Fáciles

1. **El Saludo Simple:** Crea una arrow function que reciba nombre y devuelva "Hola, [nombre]".
2. **Multiplicador:** Crea una función que reciba dos números y devuelva su producto. Si el segundo no se envía, que sea 1 por defecto.
3. **Conversor de Edad:** Función que recibe edad en años y devuelve días (x365).
4. **Es Par:** Función que recibe un número y devuelve true si es par (usa % 2 === 0), false si no.
5. **El Grito:** Función que recibe un texto y lo imprime en mayúsculas y entre signos !!!.
6. **Nombre Completo:** Función que recibe nombre y apellido y devuelve el string unido.

18.2 Nivel 2: Medios (Lógica Modular)

7. **Calculadora Arrow:** Crea funciones flecha para sumar, restar, multiplicar. Luego crea una función calcular(a, b, operacion) que ejecute la operación que le pases como callback.
8. **El Filtro de Precios:** Crea una función esBarato(precio) que devuelva true si es < 20. Usa esa función dentro de un bucle (o callback si te animas) para filtrar una

lista de precios [10, 50, 5, 80].

9. **Generador de Hashtags:** Función que recibe una frase ("Hola mundo") y devuelve "#Holamundo" (quita espacios y añade #).
10. **Scope Detector:** Escribe un código donde intentes acceder a una variable local desde fuera y explica el error en un comentario.
11. **Sumador Infinito:** Usa parámetros Rest (...numeros) para crear una función que sume TODOS los números que le pases (pueden ser 2, 5 o 100 números).
12. **Login Callback:** Función login(user, pass, onSuccess, onFailure). Si user="admin" y pass="1234", ejecuta onSuccess. Si no, onFailure. Crea los callbacks que impriman "Entrando..." o "Error".

18.3 Nivel 3: Boss Level (Integración)

13. El Procesador de Pedidos:

- Función procesarPedido(id, ...productos).
- Debe imprimir "Procesando pedido [id]".
- Si no hay productos, imprime "Carrito vacío".
- Si hay, recorre los productos e imprime "Añadido: [producto]".
- *Restricción:* Usa Arrow Function y Rest.

14. Simulador de Batalla (Callback):

- Función batalla(jugador1, jugador2, accion).
- jugador1 y jugador2 son números (vida).
- accion es una función que recibe las dos vidas y devuelve las vidas nuevas tras un ataque.
- Ejecuta batalla(100, 100, ataqueBasico) donde ataqueBasico resta 10 a ambos. Imprime resultado.

15. Transformador de Arrays (El origen de .map):

- Crea una función transformar(lista, transformador).
- Recorre la lista, aplica la función transformador a cada elemento y guarda el resultado en una NUEVA lista.

- Devuelve la nueva lista.
- Prueba: transformar([1, 2, 3], (n) => n * 10) debería devolver [10, 20, 30].

19. BIBLIOGRAFÍA

Aquí tienes los enlaces originales correspondientes a esos archivos, organizados por autor, para que puedas consultar la fuente directa:

1. Documentación Oficial y Guías (Base Teórica)

Gran parte de la estructura técnica y las definiciones precisas (especialmente del Módulo 1 y 6) provienen de la documentación de Google Developers.

- **Web.dev - Learn JavaScript:** https://web.dev/learn/javascript/

2. Midudev (JavaScript Moderno y ES2023/24)

La parte de "Modern Syntax", novedades como Object.groupBy, toSorted, async/await y buenas prácticas actuales se basan en su contenido.

- **Canal de YouTube:** midudev
- **Cursos/Web:** midu.dev

3. MoureDev (Fundamentos Sólidos)

Las explicaciones sobre tipos de datos, estructuras y la lógica inicial para principiantes se apoyan en su curso.

- **Curso JavaScript desde Cero:** Ver en YouTube
- **Web:** mouredev.com

4. SoyDalto (Nivel Junior y DOM)

Muchos de los conceptos sobre el DOM (Módulo 4), historia de JS y flujo de ejecución provienen de su curso nivel Junior.

- **Curso de JavaScript (Nivel Junior):** Ver en YouTube

5. Sergie Code (Enfoque Práctico)

Los ejemplos didácticos y el enfoque paso a paso para principiantes absolutos.

- **Curso Gratis JavaScript:** Ver en YouTube